

AI ASSIGNMENTS Q1–Q5 WITH OUTPUT

AI ASSIGNMENT - Q1 TO Q5

Generated combined assignment file with program code and actual console output.

```
=====

=====

ASSIG_Q1.PY

=====

SOURCE CODE:

"""Q1: AI-assisted Bubble Sort analysis."""

def bubble_sort(numbers):
    """Sort numbers with basic Bubble Sort and return (list, comparisons, swaps)."""
    result = numbers.copy()
    comparisons = 0
    swaps = 0

    for pass_number in range(len(result) - 1):
        for index in range(len(result) - 1 - pass_number):
            comparisons += 1
            if result[index] > result[index + 1]:
                result[index], result[index + 1] = result[index + 1], result[index]
                swaps += 1

    return result, comparisons, swaps

def bubble_sort_early_exit(numbers):
    """Bubble Sort that stops when a complete pass makes no swaps."""
    result = numbers.copy()
    comparisons = 0
    swaps = 0

    for pass_number in range(len(result) - 1):
        swapped = False
        for index in range(len(result) - 1 - pass_number):
            comparisons += 1
            if result[index] > result[index + 1]:
                result[index], result[index + 1] = result[index + 1], result[index]
                swaps += 1
        swapped = True

    if not swapped:
```

```

break

return result, comparisons, swaps

def explain_first_pass(numbers):
    """Print the adjacent comparisons and swaps in the first pass."""
    working = numbers.copy()
    print("First pass explanation:")
    for index in range(len(working) - 1):
        left = working[index]
        right = working[index + 1]
        if left > right:
            working[index], working[index + 1] = right, left
            action = f"swap -> {working}"
        else:
            action = "no swap"
        print(f"Compare {left} and {right}: {action}")
    print(f"After pass 1: {working}")

def display_results(numbers):
    """Display results for one input using both Bubble Sort versions."""
    basic = bubble_sort(numbers)
    improved = bubble_sort_early_exit(numbers)
    print(f"\nInput: {numbers}")
    print(f"Basic:    sorted={basic[0]}, comparisons={basic[1]}, swaps={basic[2]}")
    print(
        f"Improved: sorted={improved[0]}, "
        f"comparisons={improved[1]}, swaps={improved[2]}"
    )

if __name__ == "__main__":
    starter_list = [64, 34, 25, 12, 22, 11, 90]
    explain_first_pass(starter_list)

    test_lists = [
        starter_list,
        [1, 2, 3, 4, 5],
        [5, 4, 3, 2, 1],
    ]

    for test_list in test_lists:
        display_results(test_list)

```

OUTPUT:

```

First pass explanation:
Compare 64 and 34: swap -> [34, 64, 25, 12, 22, 11, 90]

```

```

Compare 64 and 25: swap -> [34, 25, 64, 12, 22, 11, 90]
Compare 64 and 12: swap -> [34, 25, 12, 64, 22, 11, 90]
Compare 64 and 22: swap -> [34, 25, 12, 22, 64, 11, 90]
Compare 64 and 11: swap -> [34, 25, 12, 22, 11, 64, 90]
Compare 64 and 90: no swap
After pass 1: [34, 25, 12, 22, 11, 64, 90]

Input: [64, 34, 25, 12, 22, 11, 90]
Basic:    sorted=[11, 12, 22, 25, 34, 64, 90], comparisons=21, swaps=14
Improved: sorted=[11, 12, 22, 25, 34, 64, 90], comparisons=21, swaps=14

Input: [1, 2, 3, 4, 5]
Basic:    sorted=[1, 2, 3, 4, 5], comparisons=10, swaps=0
Improved: sorted=[1, 2, 3, 4, 5], comparisons=4, swaps=0

Input: [5, 4, 3, 2, 1]
Basic:    sorted=[1, 2, 3, 4, 5], comparisons=10, swaps=10
Improved: sorted=[1, 2, 3, 4, 5], comparisons=10, swaps=10

```

=====

ASSIGN_Q2.PY

=====

SOURCE CODE:

```

"""Q2: Compare Linear Search and Binary Search."""

def linear_search(numbers, target):
    """Return (index, comparisons) after scanning from left to right."""
    comparisons = 0
    for index, number in enumerate(numbers):
        comparisons += 1
        if number == target:
            return index, comparisons
    return -1, comparisons

def binary_search(numbers, target):
    """Return (index, comparisons) using binary search on sorted data."""
    left = 0
    right = len(numbers) - 1
    comparisons = 0

    while left <= right:
        middle = (left + right) // 2
        comparisons += 1

        if numbers[middle] == target:
            return middle, comparisons

```

```

        if numbers[middle] < target:
            left = middle + 1
        else:
            right = middle - 1

    return -1, comparisons

def optimized_binary_search(numbers, target):
    """Use sorted range checks before running binary search."""
    if not numbers or target < numbers[0] or target > numbers[-1]:
        return -1, 0
    return binary_search(numbers, target)

def display_result(numbers, target):
    linear_result = linear_search(numbers, target)
    binary_result = binary_search(numbers, target)
    optimized_result = optimized_binary_search(numbers, target)

    print(f"Target {target}:")
    print(
        f"  Linear:    index={linear_result[0]}, "
        f"comparisons={linear_result[1]}"
    )
    print(
        f"  Binary:    index={binary_result[0]}, "
        f"comparisons={binary_result[1]}"
    )
    print(
        f"  Optimized: index={optimized_result[0]}, "
        f"comparisons={optimized_result[1]}"
    )

if __name__ == "__main__":
    numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]

    print("Why binary search needs sorted data:")
    print(
        "After checking the middle value, binary search discards half the list. "
        "That is valid only when the remaining values are ordered."
    )

    print("\nSearch results:")

    test_targets = [numbers[0], numbers[len(numbers) // 2], numbers[-1], 50, 100]
    for target in test_targets:
        display_result(numbers, target)

    print("\nUse cases:")

```

```

    print("Linear Search works on unsorted data and is suitable for small or changing lists.")
    print("Binary Search is suitable for large, sorted data when many searches are needed.")
    print("Optimization: sorted range checks immediately reject targets below the minimum or above the maximum.")

```

OUTPUT:

Why binary search needs sorted data:

After checking the middle value, binary search discards half the list. That is valid only when the remaining values are ordered.

Search results:

Target 5:

```

    Linear:    index=0, comparisons=1
    Binary:    index=0, comparisons=3
    Optimized: index=0, comparisons=3

```

Target 31:

```

    Linear:    index=4, comparisons=5
    Binary:    index=4, comparisons=1
    Optimized: index=4, comparisons=1

```

Target 89:

```

    Linear:    index=8, comparisons=9
    Binary:    index=8, comparisons=4
    Optimized: index=8, comparisons=4

```

Target 50:

```

    Linear:    index=-1, comparisons=9
    Binary:    index=-1, comparisons=3
    Optimized: index=-1, comparisons=3

```

Target 100:

```

    Linear:    index=-1, comparisons=9
    Binary:    index=-1, comparisons=4
    Optimized: index=-1, comparisons=0

```

Use cases:

Linear Search works on unsorted data and is suitable for small or changing lists.

Binary Search is suitable for large, sorted data when many searches are needed.

Optimization: sorted range checks immediately reject targets below the minimum or above the maximum.

=====

ASSIGN_Q3.PY

=====

SOURCE CODE:

```

"""Q3: Compare Bubble Sort, Selection Sort, and Insertion Sort."""

```

```

def bubble_sort(numbers):

```

```

        """Return (sorted list, comparisons, swaps)."""
        result = numbers.copy()
        comparisons = 0
        swaps = 0

        for pass_number in range(len(result) - 1):
            swapped = False
            for index in range(len(result) - 1 - pass_number):
                comparisons += 1
                if result[index] > result[index + 1]:
                    result[index], result[index + 1] = result[index + 1], result[index]
                    swaps += 1
                    swapped = True
            if not swapped:
                break

        return result, comparisons, swaps

def selection_sort(numbers):
    """Return (sorted list, comparisons, swaps)."""
    result = numbers.copy()
    comparisons = 0
    swaps = 0

    for position in range(len(result) - 1):
        smallest = position
        for index in range(position + 1, len(result)):
            comparisons += 1
            if result[index] < result[smallest]:
                smallest = index
        if smallest != position:
            result[position], result[smallest] = result[smallest], result[position]
            swaps += 1

    return result, comparisons, swaps

def insertion_sort(numbers):
    """Return (sorted list, comparisons, shifts)."""
    result = numbers.copy()
    comparisons = 0
    shifts = 0

    for position in range(1, len(result)):
        current = result[position]
        index = position - 1

        while index >= 0:
            comparisons += 1

```

```

        if result[index] <= current:
            break

        result[index + 1] = result[index]
        shifts += 1
        index -= 1

    result[index + 1] = current

    return result, comparisons, shifts

def display_result(name, result):
    sorted_data, comparisons, movements = result
    movement_name = "shifts" if name == "Insertion Sort" else "swaps"
    print(
        f"{name}: sorted={sorted_data}, comparisons={comparisons}, "
        f"{movement_name}={movements}"
    )

if __name__ == "__main__":
    data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
    expected = sorted(data)

    results = {
        "Bubble Sort": bubble_sort(data),
        "Selection Sort": selection_sort(data),
        "Insertion Sort": insertion_sort(data),
    }

    print(f"Original data: {data}")
    print("\nExperimental results using the same input:")
    for name, result in results.items():
        display_result(name, result)
        assert result[0] == expected

    print("\nStrengths and weaknesses:")
    print("Bubble Sort is simple and can stop early, but it usually performs many swaps.")
    print("Selection Sort uses few swaps, but it still makes quadratic comparisons.")
    print("Insertion Sort is simple and efficient for nearly sorted data, but shifts many values in a reversed list.")
    print("\nBest choice for nearly sorted data: Insertion Sort, because it can approach linear time.")

    print("\nTheoretical complexity:")
    print("Bubble Sort: best  $O(n)$  with early exit, average/worst  $O(n^2)$ , space  $O(1)$ .")
    print("Selection Sort: best/average/worst  $O(n^2)$ , space  $O(1)$ .")
    print("Insertion Sort: best  $O(n)$ , average/worst  $O(n^2)$ , space  $O(1)$ .")
    print("The measured counts show the expected quadratic behavior for this small unsorted input.")

```

OUTPUT:

Original data: [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]

Experimental results using the same input:

Bubble Sort: sorted=[5, 10, 12, 23, 34, 45, 56, 67, 78, 89], comparisons=44, swaps=24

Selection Sort: sorted=[5, 10, 12, 23, 34, 45, 56, 67, 78, 89], comparisons=45, swaps=6

Insertion Sort: sorted=[5, 10, 12, 23, 34, 45, 56, 67, 78, 89], comparisons=31, shifts=24

Strengths and weaknesses:

Bubble Sort is simple and can stop early, but it usually performs many swaps.

Selection Sort uses few swaps, but it still makes quadratic comparisons.

Insertion Sort is simple and efficient for nearly sorted data, but shifts many values in a reversed list.

Best choice for nearly sorted data: Insertion Sort, because it can approach linear time.

Theoretical complexity:

Bubble Sort: best $O(n)$ with early exit, average/worst $O(n^2)$, space $O(1)$.

Selection Sort: best/average/worst $O(n^2)$, space $O(1)$.

Insertion Sort: best $O(n)$, average/worst $O(n^2)$, space $O(1)$.

The measured counts show the expected quadratic behavior for this small unsorted input.

=====

ASSIGN_Q4.PY

=====

SOURCE CODE:

```
students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
    ("Esha", 88),
]

# Straightforward solution
def highest_mark_basic(students):
    """Find the highest mark by sorting a copy of the list first."""
    if not students:
        return None
    sorted_students = sorted(students, key=lambda student: student[1])
    return sorted_students[-1]

def sort_students_basic(students):
    """Return students sorted from lowest to highest mark."""
    return sorted(students, key=lambda student: student[1])

def search_student_basic(students, name):
```



```

"""Search for a student using a linear scan."""
for student_name, mark in students:
    if student_name == name:
        return (student_name, mark)
return None

# Improved solution after removing unnecessary work
def highest_mark_improved(students):
    """Find the highest-mark student in one pass, without sorting."""
    if not students:
        return None
    return max(students, key=lambda student: student[1])

def sort_students_improved(students):
    """Use Python's efficient built-in sort implementation."""
    return sorted(students, key=lambda student: student[1])

def build_student_index(students):
    """Build a dictionary for fast repeated searches by name."""
    return {name: mark for name, mark in students}

def search_student_improved(student_index, name):
    """Look up a student by name in constant average time."""
    if name not in student_index:
        return None
    return (name, student_index[name])

if __name__ == "__main__":
    student_index = build_student_index(students)

    print("Basic highest mark:", highest_mark_basic(students))
    print("Improved highest mark:", highest_mark_improved(students))
    print("Sorted students:", sort_students_improved(students))
    print("Basic search for Divya:", search_student_basic(students, "Divya"))
    print("Improved search for Divya:", search_student_improved(student_index, "Divya"))

    print("\nTime complexity comparison:")
    print("Highest mark - basic: O(n log n), improved: O(n)")
    print("Sorting - basic and improved: O(n log n)")
    print("Name search - basic: O(n) per search")
    print("Name search - improved: O(n) to build the index, then O(1) average per search")

```

OUTPUT:

```

Basic highest mark: ('Chitra', 91)
Improved highest mark: ('Chitra', 91)

```

Sorted students: [('Bala', 65), ('Divya', 74), ('Arun', 82), ('Esha', 88), ('Chitra', 91)]

Basic search for Divya: ('Divya', 74)

Improved search for Divya: ('Divya', 74)

Time complexity comparison:

Highest mark - basic: $O(n \log n)$, improved: $O(n)$

Sorting - basic and improved: $O(n \log n)$

Name search - basic: $O(n)$ per search

Name search - improved: $O(n)$ to build the index, then $O(1)$ average per search

=====

ASSIGN_Q5.PY

=====

SOURCE CODE:

"""Find the most valuable combination of items within a budget.

- Approach comparison:

- Brute force checks every subset, so it guarantees the optimum but takes $O(2^n)$ time.
- Greedy is faster, but it does not guarantee the optimum for arbitrary item prices and values.
- Dynamic programming guarantees the optimum for this 0/1 knapsack problem in $O(n * \text{budget})$ time and $O(\text{budget} * n)$ space with the selected-item tracking used below.

Dynamic programming is selected because each item can be chosen at most once and the budget is an integer.

A simple starting solution is to try every subset and keep the best valid one. For larger inputs, memory can be reduced by storing only reachable budgets in a dictionary, or the budget can be scaled to larger currency units. If the number of items is small but prices are very large, meet-in-the-middle can be used instead of a budget-sized DP table.

"""

```
items = [  
    ("Laptop", 60000),  
    ("Phone", 30000),  
    ("Tablet", 20000),  
    ("Watch", 10000),  
    ("Headphones", 5000)  
]
```

```
def best_combination(items, budget):
```

```
    """Return the maximum total price and the items used to reach it."""
```

```

    if budget < 0:
        raise ValueError("budget must be non-negative")

    # dp[i] = maximum value possible with budget i
    dp = [0] * (budget + 1)

    # Track selected items
    selected = [[] for _ in range(budget + 1)]

    for name, price in items:
        # Traverse backwards so each item is used only once
        for b in range(budget, price - 1, -1):

            if dp[b - price] + price > dp[b]:
                dp[b] = dp[b - price] + price
                selected[b] = selected[b - price] + [name]

    return dp[budget], selected[budget]

def display_results(items, budgets):
    """Print the result for each test budget."""

    for budget in budgets:
        total, combination = best_combination(items, budget)

        print("Budget:", budget)
        print("Selected Items:", combination)
        print("Total Price:", total)
        print("Remaining Budget:", budget - total)
        print("-" * 40)

if __name__ == "__main__":
    # Test with three different budgets.
    display_results(items, [50000, 70000, 100000])

```

OUTPUT:

```

Budget: 50000
Selected Items: ['Phone', 'Tablet']
Total Price: 50000
Remaining Budget: 0
-----

Budget: 70000
Selected Items: ['Laptop', 'Watch']
Total Price: 70000
Remaining Budget: 0
-----

Budget: 100000
Selected Items: ['Laptop', 'Phone', 'Watch']

```

Total Price: 100000

Remaining Budget: 0
